In [ ]:
```
# This Python 3 environment comes with many helpful analytics libraries i
# It is defined by the kaggle/python Docker image: https://github.com/kag
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) wil

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) th
# You can also write temporary files to /kaggle/temp/, but they won't be
```

In [1]:
```
import os, shutil, sys

BASE = '/kaggle/input/datasets/adityaahirwar13/renaissance-ocr-data'
EPOCH2 = '/kaggle/input/datasets/adityaahirwar13/epoch2'
WORK = '/kaggle/working/renaissance_ocr'

# create folders
for folder in ['data/raw', 'data/ground_truth', 'data/images',
               'data/crops', 'data/rodrigo', 'checkpoints', 'outputs', 'l
    os.makedirs(f'{WORK}/{folder}', exist_ok=True)

# copy python files
for f in os.listdir(BASE):
    if f.endswith(('.py', '.ipynb', '.txt')):
        shutil.copy(f'{BASE}/{f}', f'{WORK}/{f}')

# rodrigo
rodrigo_src = f'{BASE}/rodrigo/rodrigo' if os.path.exists(f'{BASE}/rodrig
shutil.copytree(rodrigo_src, f'{WORK}/data/rodrigo', dirs_exist_ok=True)

# raw PDFs
raw_src = f'{BASE}/raw/raw' if os.path.exists(f'{BASE}/raw/raw') else f'{
for f in os.listdir(raw_src):
    if os.path.isfile(f'{raw_src}/{f}'):
        shutil.copy(f'{raw_src}/{f}', f'{WORK}/data/raw/{f}')

# ground truth
gt_src = f'{BASE}/ground_truth/ground_truth' if os.path.exists(f'{BASE}/g
for f in os.listdir(gt_src):
    if os.path.isfile(f'{gt_src}/{f}'):
        shutil.copy(f'{gt_src}/{f}', f'{WORK}/data/ground_truth/{f}')

# load epoch2 adapter — check structure
print('Epoch2 dataset contents:')
for root, dirs, files in os.walk(EPOCH2):
    for f in files:
        print(' ', os.path.join(root, f).replace(EPOCH2, ''))
```

```
Epoch2 dataset contents:
  /adapter_model.safetensors
  /adapter_config.json
  /README.md
  /tokenizer.json
  /tokenizer_config.json
  /chat_template.jinja
  /processor_config.json
```

In [2]:
```
import shutil, os, subprocess

EPOCH2 = '/kaggle/input/datasets/adityaahirwar13/epoch2'
WORK   = '/kaggle/working/renaissance_ocr'

# copy epoch2 adapter to correct location
adapter_dst = f'{WORK}/checkpoints/qwen2vl_lora_renaissance'
os.makedirs(adapter_dst, exist_ok=True)
shutil.copytree(EPOCH2, adapter_dst, dirs_exist_ok=True)

print('Adapter copied. Contents:')
for f in os.listdir(adapter_dst):
    print(f'  {f}')

# install dependencies
subprocess.run(['pip', 'install', '-q',
    'transformers>=4.45.0', 'peft>=0.12.0',
    'bitsandbytes>=0.43.0', 'accelerate>=0.26.0',
    'qwen-vl-utils', 'pymupdf', 'opencv-python-headless',
], capture_output=True)
print('Dependencies installed.')
```

```
Adapter copied. Contents:
  tokenizer_config.json
  chat_template.jinja
  tokenizer.json
  processor_config.json
  adapter_model.safetensors
  adapter_config.json
  README.md
Dependencies installed.
```

In [3]:
```
import sys, os
os.chdir('/kaggle/working/renaissance_ocr')
sys.path.insert(0, '/kaggle/working/renaissance_ocr')

from dataset import convert_all_pdfs, load_all_gt, build_line_gt_map
convert_all_pdfs()
gt_map  = load_all_gt()
flat_gt = build_line_gt_map(gt_map)
print(f'GT lines: {len(flat_gt)}')
```

```
[config] env=kaggle  device=cuda
[dataset] Skip source4.pdf — already converted.
[dataset] Skip source2.pdf — already converted.
[dataset] Skip source3.pdf — already converted.
[dataset] Skip source1.pdf — already converted.
[dataset] Skip source5.pdf — already converted.
[dataset]  source4.docx: 1 page(s) GT, 25 lines, 8 paleographer notes
[dataset]  source1.docx: 1 page(s) GT, 24 lines, 8 paleographer notes
[dataset]  source3.docx: 1 page(s) GT, 21 lines, 8 paleographer notes
[dataset]  source5.docx: 1 page(s) GT, 24 lines, 8 paleographer notes
[dataset]  source2.docx: 1 page(s) GT, 33 lines, 8 paleographer notes
[dataset] GT loaded for 5 source(s).
GT lines: 127
```

In [4]:

```
resume_code = '''
import os, json, random
from pathlib import Path
import torch
from torch.optim import AdamW
from transformers import get_linear_schedule_with_warmup, AutoProcessor
from qwen_vl_utils import process_vision_info
from tqdm import tqdm
import shutil
from peft import PeftModel

from config import (
    CHECKPOINTS_DIR, LEARNING_RATE,
    LOGGING_STEPS, SEED, LORA_ADAPTER,
    TRANSCRIBE_PROMPT, MAX_SEQ_LEN,
)
from dataset import HandwrittenLineDataset, split_dataset
from model import load_base_model

torch.manual_seed(SEED)

KAGGLE_OUT   = Path("/kaggle/working")
START_EPOCH  = 2   # already done
RESUME_EPOCHS = 3  # run epochs 3, 4, 5


def save_adapter_safely(model, processor, tag):
    model.save_pretrained(str(LORA_ADAPTER))
    processor.save_pretrained(str(LORA_ADAPTER))
    out_dir  = KAGGLE_OUT / f"adapter_{tag}"
    model.save_pretrained(str(out_dir))
    processor.save_pretrained(str(out_dir))
    zip_path = str(KAGGLE_OUT / f"adapter_{tag}")
    shutil.make_archive(zip_path, "zip", str(out_dir))
    size = Path(zip_path + ".zip").stat().st_size / 1e6
    print(f"[save] adapter_{tag}.zip ({size:.1f} MB)")
    print(f">>> DOWNLOAD adapter_{tag}.zip FROM OUTPUT PANEL NOW <<<")


def process_single_sample(processor, image, gt_text, device):
    messages = [
        {
            "role": "user",
            "content": [
                {"type": "image", "image": image},
                {"type": "text",  "text":  TRANSCRIBE_PROMPT},
```

```
            ],
        },
        {"role": "assistant", "content": gt_text},
    ]
    text = processor.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=False
    )
    image_inputs, _ = process_vision_info(messages)
    inputs = processor(
        text=[text],
        images=image_inputs,
        return_tensors="pt",
        padding=False,
        truncation=True,
        max_length=MAX_SEQ_LEN,
    ).to(device)

    input_ids  = inputs["input_ids"][0]
    labels     = input_ids.clone()
    prompt_len = len(processor.tokenizer(
        processor.apply_chat_template(
            messages[:-1], tokenize=False, add_generation_prompt=True
        ),
        truncation=True, max_length=MAX_SEQ_LEN
    ).input_ids)
    labels[:prompt_len] = -100
    inputs["labels"]    = labels.unsqueeze(0)
    return inputs


def resume_train():
    print("=" * 60)
    print(f" Resuming from epoch {START_EPOCH+1} to 5")
    print("=" * 60)

    device = torch.device("cuda:0")

    # load base + epoch2 adapter
    base_model, processor = load_base_model(quantize=True)
    print(f"[train] Loading epoch {START_EPOCH} adapter...")
    model = PeftModel.from_pretrained(base_model, str(LORA_ADAPTER))
    model.train()

    # make sure LoRA params are trainable
    for name, param in model.named_parameters():
        if "lora" in name.lower():
            param.requires_grad = True

    trainable = sum(p.numel() for p in model.parameters() if p.requires_g
    print(f"[train] Trainable params: {trainable:,}")

    # dataset
    full_dataset = HandwrittenLineDataset(augment=True)
    train_ds, _  = split_dataset(full_dataset, val_ratio=0.0)
    print(f"[train] Samples: {len(train_ds)}")

    # optimizer with half LR — already partially trained
    optimizer = AdamW(
        [p for p in model.parameters() if p.requires_grad],
        lr=LEARNING_RATE * 0.5,
```

```
    )
    total_steps = (len(train_ds) // 8) * RESUME_EPOCHS
    scheduler   = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps   = 5,
        num_training_steps = total_steps,
    )

    history     = {"train_loss": [], "steps": []}
    global_step = 0
    GRAD_ACCUM  = 8

    for epoch in range(RESUME_EPOCHS):
        actual_epoch = START_EPOCH + epoch + 1
        print(f"\\n[train] Epoch {actual_epoch}/5")
        epoch_loss  = 0.0
        valid_count = 0
        optimizer.zero_grad()

        indices = list(range(len(train_ds)))
        random.shuffle(indices)

        for i, idx in enumerate(tqdm(indices, desc=f"Epoch {actual_epoch}
            sample = train_ds[idx]
            try:
                inputs  = process_single_sample(
                    processor, sample["image"], sample["text"], device
                )
                outputs = model(**inputs)
                loss    = outputs.loss / GRAD_ACCUM

                if loss is None or torch.isnan(loss):
                    continue

                loss.backward()
                epoch_loss  += loss.item() * GRAD_ACCUM
                valid_count += 1

            except Exception:
                continue

            if (i + 1) % GRAD_ACCUM == 0:
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()
                scheduler.step()
                optimizer.zero_grad()
                global_step += 1

                if global_step % LOGGING_STEPS == 0:
                    avg = epoch_loss / max(valid_count, 1)
                    print(f"  step={global_step}  loss={avg:.4f}")
                    history["train_loss"].append(avg)
                    history["steps"].append(global_step)

        avg = epoch_loss / max(valid_count, 1)
        print(f"[train] Epoch {actual_epoch} avg loss: {avg:.4f}")
        save_adapter_safely(model, processor, tag=f"epoch{actual_epoch}")

    save_adapter_safely(model, processor, tag="final")
    print("[train] Resume complete.")
```

```
    return history
'''

with open('/kaggle/working/renaissance_ocr/resume_train.py', 'w') as f:
    f.write(resume_code)
print('resume_train.py written.')
```

```
resume_train.py written.
```

In [ ]:

```
import os, sys
os.environ['CUDA_VISIBLE_DEVICES'] = '0'
os.chdir('/kaggle/working/renaissance_ocr')
sys.path.insert(0, '/kaggle/working/renaissance_ocr')

for mod in list(sys.modules.keys()):
    if mod in ['train', 'resume_train', 'model', 'dataset', 'config']:
        del sys.modules[mod]

from resume_train import resume_train
history = resume_train()
```

```
[config] env=kaggle  device=cuda
============================================================
 Resuming from epoch 3 to 5
============================================================
[model] Loading Qwen/Qwen2-VL-2B-Instruct …
Warning: You are sending unauthenticated requests to the HF Hub. Please se
t a HF_TOKEN to enable higher rate limits and faster downloads.
The image processor of type `Qwen2VLImageProcessor` is now loaded as a fas
t processor by default, even if the model checkpoint was saved with a slow
processor. This is a breaking change and may produce slightly different ou
tputs. To continue using the slow processor, instantiate this class with `
use_fast=False`.
Downloading (incomplete total...): 0.00B [00:00, ?B/s]
Fetching 2 files:   0%|          | 0/2 [00:00<?, ?it/s]
Loading weights:   0%|          | 0/729 [00:00<?, ?it/s]
[model] Base model loaded. Parameters: 1.22B
[train] Loading epoch 2 adapter...
[train] Trainable params: 18,464,768
[dataset]  source4.docx: 1 page(s) GT, 25 lines, 8 paleographer notes
[dataset]  source1.docx: 1 page(s) GT, 24 lines, 8 paleographer notes
[dataset]  source3.docx: 1 page(s) GT, 21 lines, 8 paleographer notes
[dataset]  source5.docx: 1 page(s) GT, 24 lines, 8 paleographer notes
[dataset]  source2.docx: 1 page(s) GT, 33 lines, 8 paleographer notes
[dataset] GT loaded for 5 source(s).
[dataset] RenAIssance GT crops matched: 0
[dataset] Rodrigo train split: 9000 stems
[dataset] Rodrigo transcriptions loaded: 15010
[dataset] Rodrigo: 9000 train samples loaded. (0 images missing — normal i
f small)
[dataset] Total samples: 9000
[dataset] Train: 9000 (-1 renaissance + 9000 rodrigo)
[dataset] Val:   0 (renaissance only)
[train] Samples: 9000

[train] Epoch 3/5
Epoch 3:   1%|          | 80/9000 [00:48<1:32:46,  1.60it/s]
  step=10  loss=0.0285
Epoch 3:   2%||          | 160/9000 [01:41<1:39:22,  1.48it/s]
  step=20  loss=0.0300
```

```
Epoch 3:   3%|▌          | 240/9000 [02:32<1:34:54,  1.54it/s]
  step=30  loss=0.0304
Epoch 3:   4%|▌          | 320/9000 [03:24<1:34:15,  1.53it/s]
  step=40  loss=0.0300
Epoch 3:   4%|▌          | 400/9000 [04:16<1:34:20,  1.52it/s]
  step=50  loss=0.0309
Epoch 3:   5%|▌          | 480/9000 [05:08<1:33:10,  1.52it/s]
  step=60  loss=0.0311
Epoch 3:   6%|▌          | 560/9000 [06:00<1:32:49,  1.52it/s]
  step=70  loss=0.0314
Epoch 3:   7%|▌          | 640/9000 [06:52<1:30:40,  1.54it/s]
  step=80  loss=0.0315
Epoch 3:   8%|▌          | 720/9000 [07:43<1:29:57,  1.53it/s]
  step=90  loss=0.0312
Epoch 3:   9%|▌          | 800/9000 [08:36<1:30:08,  1.52it/s]
  step=100  loss=0.0308
Epoch 3:  10%|▌          | 880/9000 [09:28<1:28:43,  1.53it/s]
  step=110  loss=0.0307
Epoch 3:  11%|▌          | 960/9000 [10:20<1:28:55,  1.51it/s]
  step=120  loss=0.0309
Epoch 3:  12%|▌          | 1040/9000 [11:12<1:26:24,  1.54it/s]
  step=130  loss=0.0309
Epoch 3:  12%|▌          | 1120/9000 [12:04<1:25:50,  1.53it/s]
  step=140  loss=0.0301
Epoch 3:  13%|▌          | 1200/9000 [12:56<1:25:08,  1.53it/s]
  step=150  loss=0.0299
Epoch 3:  14%|▌          | 1280/9000 [13:48<1:25:00,  1.51it/s]
  step=160  loss=0.0296
Epoch 3:  15%|▌          | 1360/9000 [14:40<1:22:58,  1.53it/s]
  step=170  loss=0.0294
Epoch 3:  16%|▌          | 1440/9000 [15:32<1:21:43,  1.54it/s]
  step=180  loss=0.0293
Epoch 3:  17%|▌          | 1520/9000 [16:24<1:20:33,  1.55it/s]
  step=190  loss=0.0290
Epoch 3:  18%|▌          | 1600/9000 [17:15<1:20:24,  1.53it/s]
  step=200  loss=0.0286
Epoch 3:  19%|▌          | 1680/9000 [18:07<1:18:33,  1.55it/s]
  step=210  loss=0.0286
Epoch 3:  20%|▌          | 1760/9000 [18:59<1:18:43,  1.53it/s]
  step=220  loss=0.0285
Epoch 3:  20%|▌          | 1840/9000 [19:50<1:17:50,  1.53it/s]
  step=230  loss=0.0286
Epoch 3:  21%|▌          | 1920/9000 [20:42<1:17:10,  1.53it/s]
  step=240  loss=0.0286
Epoch 3:  22%|▌          | 2000/9000 [21:33<1:13:15,  1.59it/s]
  step=250  loss=0.0284
Epoch 3:  23%|▌          | 2080/9000 [22:25<1:15:46,  1.52it/s]
  step=260  loss=0.0281
Epoch 3:  24%|▌          | 2160/9000 [23:17<1:14:33,  1.53it/s]
  step=270  loss=0.0281
Epoch 3:  25%|▌          | 2240/9000 [24:08<1:13:53,  1.52it/s]
  step=280  loss=0.0278
```

```
Epoch 3:  26%|██        | 2320/9000 [25:00<1:12:41,  1.53it/s]
  step=290  loss=0.0277
Epoch 3:  27%|██        | 2400/9000 [25:51<1:11:57,  1.53it/s]
  step=300  loss=0.0278
Epoch 3:  28%|██        | 2480/9000 [26:43<1:10:56,  1.53it/s]
  step=310  loss=0.0277
Epoch 3:  28%|██        | 2560/9000 [27:35<1:10:00,  1.53it/s]
  step=320  loss=0.0275
Epoch 3:  29%|██        | 2640/9000 [28:26<1:08:05,  1.56it/s]
  step=330  loss=0.0276
Epoch 3:  30%|██        | 2720/9000 [29:18<1:07:52,  1.54it/s]
  step=340  loss=0.0276
Epoch 3:  31%|██        | 2800/9000 [30:10<1:07:48,  1.52it/s]
  step=350  loss=0.0277
Epoch 3:  32%|██        | 2880/9000 [31:01<1:06:13,  1.54it/s]
  step=360  loss=0.0277
Epoch 3:  33%|██        | 2960/9000 [31:53<1:07:36,  1.49it/s]
  step=370  loss=0.0277
Epoch 3:  34%|██        | 3040/9000 [32:45<1:04:50,  1.53it/s]
  step=380  loss=0.0275
Epoch 3:  35%|██        | 3120/9000 [33:36<1:00:59,  1.61it/s]
  step=390  loss=0.0275
Epoch 3:  36%|██        | 3200/9000 [34:28<1:02:19,  1.55it/s]
  step=400  loss=0.0274
Epoch 3:  36%|└──       | 3280/9000 [35:20<1:02:26,  1.53it/s]
  step=410  loss=0.0275
Epoch 3:  37%|└──       | 3360/9000 [36:11<1:00:40,  1.55it/s]
  step=420  loss=0.0276
Epoch 3:  38%|└──       | 3440/9000 [37:03<58:01,  1.60it/s]
  step=430  loss=0.0276
Epoch 3:  39%|└──       | 3520/9000 [37:55<58:46,  1.55it/s]
  step=440  loss=0.0275
Epoch 3:  40%|└──       | 3600/9000 [38:47<58:46,  1.53it/s]
  step=450  loss=0.0274
Epoch 3:  41%|└──       | 3680/9000 [39:38<57:30,  1.54it/s]
  step=460  loss=0.0273
Epoch 3:  42%|└──       | 3760/9000 [40:29<56:19,  1.55it/s]
  step=470  loss=0.0273
Epoch 3:  43%|└──       | 3840/9000 [41:21<57:10,  1.50it/s]
  step=480  loss=0.0274
Epoch 3:  44%|└──       | 3920/9000 [42:13<54:47,  1.55it/s]
  step=490  loss=0.0273
Epoch 3:  44%|└──       | 4000/9000 [43:04<54:14,  1.54it/s]
  step=500  loss=0.0273
Epoch 3:  45%|└──       | 4080/9000 [43:56<53:31,  1.53it/s]
  step=510  loss=0.0273
Epoch 3:  46%|└──       | 4160/9000 [44:47<51:56,  1.55it/s]
  step=520  loss=0.0271
Epoch 3:  47%|└──       | 4240/9000 [45:39<50:48,  1.56it/s]
  step=530  loss=0.0270
Epoch 3:  48%|└──       | 4320/9000 [46:31<50:54,  1.53it/s]
  step=540  loss=0.0270
```

```
Epoch 3:  49%|▌▌▌▌▌      | 4400/9000 [47:22<48:44,  1.57it/s]
  step=550  loss=0.0270
Epoch 3:  50%|▌▌▌▌▌      | 4480/9000 [48:14<49:20,  1.53it/s]
  step=560  loss=0.0270
Epoch 3:  51%|▌▌▌▌▌      | 4560/9000 [49:06<48:27,  1.53it/s]
  step=570  loss=0.0269
Epoch 3:  52%|▌▌▌▌▌      | 4640/9000 [49:58<47:07,  1.54it/s]
  step=580  loss=0.0268
Epoch 3:  52%|▌▌▌▌▌      | 4720/9000 [50:49<45:37,  1.56it/s]
  step=590  loss=0.0268
Epoch 3:  53%|▌▌▌▌▌      | 4800/9000 [51:41<45:44,  1.53it/s]
  step=600  loss=0.0268
Epoch 3:  54%|▌▌▌▌▌      | 4880/9000 [52:32<44:43,  1.54it/s]
  step=610  loss=0.0267
Epoch 3:  55%|▌▌▌▌▌      | 4960/9000 [53:24<43:51,  1.54it/s]
  step=620  loss=0.0266
Epoch 3:  56%|▌▌▌▌▌      | 5040/9000 [54:16<43:33,  1.52it/s]
  step=630  loss=0.0265
Epoch 3:  57%|▌▌▌▌▌      | 5120/9000 [55:08<42:22,  1.53it/s]
  step=640  loss=0.0264
Epoch 3:  58%|▌▌▌▌▌      | 5200/9000 [56:00<41:26,  1.53it/s]
  step=650  loss=0.0264
Epoch 3:  59%|▌▌▌▌▌      | 5280/9000 [56:51<39:08,  1.58it/s]
  step=660  loss=0.0264
Epoch 3:  60%|▌▌▌▌▌      | 5360/9000 [57:43<39:44,  1.53it/s]
  step=670  loss=0.0263
Epoch 3:  60%|▌▌▌▌▌      | 5440/9000 [58:35<38:37,  1.54it/s]
  step=680  loss=0.0262
Epoch 3:  61%|▌▌▌▌▌      | 5520/9000 [59:26<37:53,  1.53it/s]
  step=690  loss=0.0262
Epoch 3:  62%|▌▌▌▌▌      | 5600/9000 [1:00:18<37:02,  1.53it/s]
  step=700  loss=0.0261
Epoch 3:  63%|▌▌▌▌▌      | 5680/9000 [1:01:09<36:40,  1.51it/s]
  step=710  loss=0.0260
Epoch 3:  64%|▌▌▌▌▌      | 5760/9000 [1:02:01<34:49,  1.55it/s]
  step=720  loss=0.0260
Epoch 3:  65%|▌▌▌▌▌      | 5840/9000 [1:02:53<33:08,  1.59it/s]
  step=730  loss=0.0259
Epoch 3:  66%|▌▌▌▌▌      | 5920/9000 [1:03:45<33:38,  1.53it/s]
  step=740  loss=0.0259
Epoch 3:  67%|▌▌▌▌▌      | 6000/9000 [1:04:37<32:31,  1.54it/s]
  step=750  loss=0.0260
Epoch 3:  68%|▌▌▌▌▌      | 6080/9000 [1:05:29<31:16,  1.56it/s]
  step=760  loss=0.0260
Epoch 3:  68%|▌▌▌▌▌      | 6160/9000 [1:06:20<30:50,  1.53it/s]
  step=770  loss=0.0258
Epoch 3:  69%|▌▌▌▌▌      | 6240/9000 [1:07:11<29:58,  1.53it/s]
  step=780  loss=0.0257
Epoch 3:  70%|▌▌▌▌▌      | 6320/9000 [1:08:04<29:24,  1.52it/s]
  step=790  loss=0.0257
Epoch 3:  71%|▌▌▌▌▌      | 6400/9000 [1:08:55<28:10,  1.54it/s]
  step=800  loss=0.0257
```

```
Epoch 3:  72%|▉▉▉▉▉▉▉▉   | 6480/9000 [1:09:47<27:03,  1.55it/s]
  step=810  loss=0.0256
Epoch 3:  73%|▉▉▉▉▉▉▉▉   | 6560/9000 [1:10:38<25:27,  1.60it/s]
  step=820  loss=0.0255
Epoch 3:  74%|▉▉▉▉▉▉▉▉   | 6640/9000 [1:11:29<25:58,  1.51it/s]
  step=830  loss=0.0254
Epoch 3:  75%|▉▉▉▉▉▉▉▉   | 6720/9000 [1:12:21<24:45,  1.53it/s]
  step=840  loss=0.0254
Epoch 3:  76%|▉▉▉▉▉▉▉▉   | 6800/9000 [1:13:13<23:57,  1.53it/s]
  step=850  loss=0.0253
Epoch 3:  76%|▉▉▉▉▉▉▉▉   | 6880/9000 [1:14:04<23:01,  1.53it/s]
  step=860  loss=0.0253
Epoch 3:  77%|▉▉▉▉▉▉▉▉   | 6960/9000 [1:14:56<22:08,  1.54it/s]
  step=870  loss=0.0252
Epoch 3:  78%|▉▉▉▉▉▉▉▉   | 7040/9000 [1:15:48<21:10,  1.54it/s]
  step=880  loss=0.0251
Epoch 3:  79%|▉▉▉▉▉▉▉▉   | 7120/9000 [1:16:39<20:32,  1.53it/s]
  step=890  loss=0.0251
Epoch 3:  80%|▉▉▉▉▉▉▉▉   | 7200/9000 [1:17:31<19:10,  1.56it/s]
  step=900  loss=0.0250
Epoch 3:  81%|▉▉▉▉▉▉▉▉   | 7280/9000 [1:18:23<17:58,  1.59it/s]
  step=910  loss=0.0249
Epoch 3:  82%|▉▉▉▉▉▉▉▉   | 7360/9000 [1:19:15<17:53,  1.53it/s]
  step=920  loss=0.0249
Epoch 3:  83%|▉▉▉▉▉▉▉▉   | 7440/9000 [1:20:06<17:00,  1.53it/s]
  step=930  loss=0.0249
Epoch 3:  84%|▉▉▉▉▉▉▉▉   | 7520/9000 [1:20:58<16:02,  1.54it/s]
  step=940  loss=0.0248
Epoch 3:  84%|▉▉▉▉▉▉▉▉   | 7600/9000 [1:21:49<15:09,  1.54it/s]
  step=950  loss=0.0248
Epoch 3:  85%|▉▉▉▉▉▉▉▉   | 7680/9000 [1:22:41<14:23,  1.53it/s]
  step=960  loss=0.0247
Epoch 3:  86%|▉▉▉▉▉▉▉▉   | 7760/9000 [1:23:33<13:28,  1.53it/s]
  step=970  loss=0.0247
Epoch 3:  87%|▉▉▉▉▉▉▉▉   | 7840/9000 [1:24:25<12:33,  1.54it/s]
  step=980  loss=0.0246
Epoch 3:  88%|▉▉▉▉▉▉▉▉   | 7920/9000 [1:25:16<11:43,  1.53it/s]
  step=990  loss=0.0246
Epoch 3:  89%|▉▉▉▉▉▉▉▉   | 8000/9000 [1:26:08<12:00,  1.39it/s]
  step=1000  loss=0.0245
Epoch 3:  90%|▉▉▉▉▉▉▉▉   | 8080/9000 [1:27:00<09:56,  1.54it/s]
  step=1010  loss=0.0245
Epoch 3:  91%|▉▉▉▉▉▉▉▉   | 8160/9000 [1:27:51<09:08,  1.53it/s]
  step=1020  loss=0.0245
Epoch 3:  92%|▉▉▉▉▉▉▉▉   | 8240/9000 [1:28:43<08:11,  1.55it/s]
  step=1030  loss=0.0244
Epoch 3:  92%|▉▉▉▉▉▉▉▉   | 8320/9000 [1:29:34<07:25,  1.53it/s]
  step=1040  loss=0.0244
Epoch 3:  93%|▉▉▉▉▉▉▉▉   | 8400/9000 [1:30:26<06:31,  1.53it/s]
  step=1050  loss=0.0244
Epoch 3:  94%|▉▉▉▉▉▉▉▉   | 8480/9000 [1:31:18<05:38,  1.54it/s]
  step=1060  loss=0.0243
```

```
Epoch 3:  95%|██████████| 8560/9000 [1:32:10<04:51,  1.51it/s]
  step=1070  loss=0.0242
Epoch 3:  96%|██████████| 8640/9000 [1:33:02<03:49,  1.57it/s]
  step=1080  loss=0.0242
Epoch 3:  97%|██████████| 8720/9000 [1:33:54<03:02,  1.53it/s]
  step=1090  loss=0.0241
Epoch 3:  98%|██████████| 8800/9000 [1:34:45<02:10,  1.53it/s]
  step=1100  loss=0.0241
Epoch 3:  99%|██████████| 8880/9000 [1:35:37<01:21,  1.48it/s]
  step=1110  loss=0.0240
Epoch 3: 100%|██████████| 8960/9000 [1:36:29<00:26,  1.54it/s]
  step=1120  loss=0.0240
Epoch 3: 100%|██████████| 9000/9000 [1:36:55<00:00,  1.55it/s]
[train] Epoch 3 avg loss: 0.0239
[save] adapter_epoch3.zip (70.6 MB)
>>> DOWNLOAD adapter_epoch3.zip FROM OUTPUT PANEL NOW <<<

[train] Epoch 4/5
Epoch 4:   0%|          | 40/9000 [00:25<1:37:44,  1.53it/s]
  step=1130  loss=0.0151
Epoch 4:   1%||          | 120/9000 [01:18<1:36:20,  1.54it/s]
  step=1140  loss=0.0148
Epoch 4:   2%||          | 200/9000 [02:09<1:36:26,  1.52it/s]
  step=1150  loss=0.0158
Epoch 4:   3%|▌          | 280/9000 [03:01<1:34:32,  1.54it/s]
  step=1160  loss=0.0141
Epoch 4:   4%|▌          | 360/9000 [03:53<1:33:48,  1.54it/s]
  step=1170  loss=0.0138
Epoch 4:   5%|▌          | 440/9000 [04:44<1:32:45,  1.54it/s]
  step=1180  loss=0.0136
Epoch 4:   6%|▌          | 520/9000 [05:36<1:32:18,  1.53it/s]
  step=1190  loss=0.0133
Epoch 4:   7%|▌          | 600/9000 [06:28<1:28:37,  1.58it/s]
  step=1200  loss=0.0132
Epoch 4:   8%|▌          | 680/9000 [07:20<1:31:10,  1.52it/s]
  step=1210  loss=0.0131
Epoch 4:   8%|▌          | 760/9000 [08:11<1:29:23,  1.54it/s]
  step=1220  loss=0.0131
Epoch 4:   9%|▌          | 840/9000 [09:03<1:27:10,  1.56it/s]
  step=1230  loss=0.0132
Epoch 4:  10%|▌          | 920/9000 [09:54<1:27:29,  1.54it/s]
  step=1240  loss=0.0131
Epoch 4:  11%|▌          | 1000/9000 [10:46<1:27:13,  1.53it/s]
  step=1250  loss=0.0131
Epoch 4:  12%|▌          | 1080/9000 [11:38<1:26:49,  1.52it/s]
  step=1260  loss=0.0130
Epoch 4:  13%|▌          | 1160/9000 [12:29<1:23:39,  1.56it/s]
  step=1270  loss=0.0129
Epoch 4:  14%|▌          | 1240/9000 [13:21<1:23:56,  1.54it/s]
  step=1280  loss=0.0125
Epoch 4:  15%|▌          | 1320/9000 [14:13<1:23:07,  1.54it/s]
  step=1290  loss=0.0124
Epoch 4:  16%|▌          | 1400/9000 [15:04<1:22:21,  1.54it/s]
```

```
  step=1300  loss=0.0122
Epoch 4:  16%|▉         | 1480/9000 [15:56<1:21:23,  1.54it/s]
  step=1310  loss=0.0123
Epoch 4:  17%|▉         | 1560/9000 [16:48<1:21:25,  1.52it/s]
  step=1320  loss=0.0124
Epoch 4:  18%|▉         | 1640/9000 [17:40<1:20:06,  1.53it/s]
  step=1330  loss=0.0122
Epoch 4:  19%|▉         | 1720/9000 [18:31<1:16:53,  1.58it/s]
  step=1340  loss=0.0121
Epoch 4:  20%|▉         | 1800/9000 [19:23<1:18:13,  1.53it/s]
  step=1350  loss=0.0122
Epoch 4:  21%|▉         | 1880/9000 [20:15<1:16:39,  1.55it/s]
  step=1360  loss=0.0121
Epoch 4:  22%|▉         | 1960/9000 [21:06<1:16:40,  1.53it/s]
  step=1370  loss=0.0122
Epoch 4:  23%|▉         | 2040/9000 [21:58<1:15:40,  1.53it/s]
  step=1380  loss=0.0123
Epoch 4:  24%|▉         | 2120/9000 [22:50<1:15:04,  1.53it/s]
  step=1390  loss=0.0123
Epoch 4:  24%|▉         | 2200/9000 [23:41<1:14:15,  1.53it/s]
  step=1400  loss=0.0122
Epoch 4:  25%|▉         | 2280/9000 [24:33<1:24:08,  1.33it/s]
  step=1410  loss=0.0123
Epoch 4:  26%|▉         | 2360/9000 [25:25<1:12:10,  1.53it/s]
  step=1420  loss=0.0122
Epoch 4:  27%|▉         | 2440/9000 [26:17<1:12:12,  1.51it/s]
  step=1430  loss=0.0122
Epoch 4:  30%|▉         | 2680/9000 [28:52<1:09:00,  1.53it/s]
  step=1460  loss=0.0120
Epoch 4:  31%|▉         | 2760/9000 [29:44<1:07:02,  1.55it/s]
  step=1470  loss=0.0120
Epoch 4:  32%|▉         | 2840/9000 [30:35<1:04:49,  1.58it/s]
  step=1480  loss=0.0119
Epoch 4:  32%|▉         | 2920/9000 [31:27<1:06:28,  1.52it/s]
  step=1490  loss=0.0120
Epoch 4:  33%|▉         | 3000/9000 [32:19<1:03:00,  1.59it/s]
  step=1500  loss=0.0120
Epoch 4:  34%|▉         | 3080/9000 [33:10<1:04:39,  1.53it/s]
  step=1510  loss=0.0119
Epoch 4:  35%|▉         | 3160/9000 [34:02<1:07:10,  1.45it/s]
  step=1520  loss=0.0120
Epoch 4:  36%|▉         | 3240/9000 [34:53<1:02:48,  1.53it/s]
  step=1530  loss=0.0119
Epoch 4:  37%|▉         | 3320/9000 [35:44<1:00:11,  1.57it/s]
  step=1540  loss=0.0118
Epoch 4:  38%|▉         | 3400/9000 [36:36<1:00:54,  1.53it/s]
  step=1550  loss=0.0119
Epoch 4:  39%|▉         | 3480/9000 [37:27<59:53,  1.54it/s]
  step=1560  loss=0.0120
Epoch 4:  40%|▉         | 3560/9000 [38:19<59:16,  1.53it/s]
  step=1570  loss=0.0120
Epoch 4:  40%|▉         | 3640/9000 [39:10<57:54,  1.54it/s]
```

```
  step=1580  loss=0.0120
Epoch 4:  41%|▉▉▉▉      | 3720/9000 [40:02<56:29,  1.56it/s]
  step=1590  loss=0.0119
Epoch 4:  42%|▉▉▉▉      | 3800/9000 [40:53<56:06,  1.54it/s]
  step=1600  loss=0.0119
Epoch 4:  43%|▉▉▉▉      | 3880/9000 [41:45<55:13,  1.55it/s]
  step=1610  loss=0.0119
Epoch 4:  44%|▉▉▉▉      | 3960/9000 [42:36<54:21,  1.55it/s]
  step=1620  loss=0.0118
Epoch 4:  45%|▉▉▉▉      | 4040/9000 [43:28<54:20,  1.52it/s]
  step=1630  loss=0.0118
Epoch 4:  46%|▉▉▉▉      | 4120/9000 [44:20<53:23,  1.52it/s]
  step=1640  loss=0.0118
Epoch 4:  46%|▉▉▉▉      | 4149/9000 [44:39<52:47,  1.53it/s]
```

In [ ]: